Practical 4: Implementation and complexity analysis of algorithms based on dynamic programming. Floyd–Warshall Algorithm (All-Pairs Shortest Paths)

Floyd-Warshall Algorithm

Objective

- Unlike single-source algorithms like Dijkstra or Bellman-Ford, it determines the optimal distance between all possible start and end nodes simultaneously.
- It can correctly compute shortest paths in graphs containing negative edge weights, provided the graph contains no negative cycles.
- t systematically improves path estimates by considering each vertex as a potential intermediate pivot to find shorter routes between other nodes.

Theory

The Floyd-Warshall algorithm is a graph algorithm that is deployed to find the shortest path between all the vertices present in a weighted graph. This algorithm is different from other shortest path algorithms; to describe it simply, this algorithm uses each vertex in the graph as a pivot to check if it provides the shortest way to travel from one point to another.

There are mainly four steps in the algorithm:

- Construct an adjacency matrix A with all the costs of edges present in the graph. If there is no path between two vertices, mark the value as ∞
- Derive another adjacency matrix A_1 from A by keeping the first row and the first column of the original adjacency matrix unchanged. For the remaining entries, denoted by $A_1[i, j]$, update them according to the following rule:

$$A_1[i,j] = \begin{cases} A[i,1] + A[1,j], & \text{if } A[i,j] > A[i,1] + A[1,j], \\ A[i,j], & \text{otherwise.} \end{cases}$$

  In this step, the pivot vertex is the first vertex, i.e., $k = 1$.
- Repeat Step 2 for all the vertices in the graph by changing the k value for every pivot vertex until the final matrix is achieved.
- The final adjacency matrix obtained is the final solution with all the shortest paths.

Algorithm

Algorithm 1 Quick Sort

```
1: procedure FloydWarshall(W, n)
2:     D ← W
3:     for k ← 0 to n − 1 do
4:         for i ← 0 to n − 1 do
5:             for j ← 0 to n − 1 do
6:                 if D[i][k] + D[k][j] < D[i][j] then
7:                     D[i][j] ← D[i][k] + D[k][j]
8:                 end if
9:             end for
10:        end for
11:    end for
12:    return D
13: end procedure
```

Python Code

```python
def Floyd_warsall(w, n):
    D = [row[:] for row in w]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if D[i][k] + D[k][j] < D[i][j]:
                    D[i][j] = D[i][k] + D[k][j]
    return D
```

Output

```python
INF = float('inf')
n = 4
W = [
    [0, INF, 3, INF],
    [2, 0, INF, INF],
    [INF, 7, 0, 1],
    [6, INF, INF, 0]
]

d = Floyd_warsall(W, n)
for i in range(n):
    for j in range(n):
        print(d[i][j], end='\t')
    print()
```

```
0       10      3       4
2       0       5       6
7       7       0       1
6       16      9       0
```

Complexity Analysis

Time Complexity:

- $O(n^3)$

Space Complexity:

- $O(n^2)$